HOMEWORK A - 2026

Introduzione a Java
Testing

Esercizio 1 (Testing)

- Scrivere le classi di test JUnit per le classi **`Stanza`**, **`Partita`**
- Scrivere almeno tre metodi di test per ciascuno dei metodi più importanti di queste classi
- Perseguire la qualità dei test, in particolare la loro *minimalità*. Allo scopo scrivere test-case
  - brevi e coincisi: che utilizzino meno oggetti possibile ed in uno stato iniziale più semplice possibile
  - fattorizzati: senza codice duplicato ma evitando di comprometterne leggibilità ed autocontenimento
  - indipendenti: che evitino di ripetere scenari di test già coperti da altri test-case

Esercizio 2 (Refactoring)

- Introdurre la classe **`Labirinto`**
  - ha la responsabilità di creare il labirinto, di memorizzare la stanza iniziale (entrata) e quella finale (uscita)
  - aggiungere un riferimento ad un'istanza di **`Labirinto`** nella classe **`Partita`** (che ovviamente dovrà essere liberata dalle responsabilità spostate nella nuova classe)
- Introdurre la classe **`Giocatore`** e la classe **`Borsa`**
  - **`Giocatore`** ha la responsabilità di gestire i CFU del giocatore e di memorizzare gli attrezzi in un oggetto istanza della classe **`Borsa`** >>(vedi codice a seguire)
  - aggiungere un riferimento ad un'istanza di **`Giocatore`** nella classe **`Partita`** (che ovviamente dovrà essere liberata dalle responsabilità spostate nella nuova classe)
- Nell'ordine:
  - scrivere classi di test JUnit per **`Giocatore`**, **`Borsa`** e **`Labirinto`**
  - introdurre le classi **`Labirinto`** e **`Giocatore`** e **`Borsa`** nel codice

Esercizio 3 (Refactoring, Nuovi Comandi)

- Modificare il gioco affinché il giocatore possa "prendere" e "posare" degli attrezzi
- Per implementare questa modifica è necessario fare le seguenti operazioni:
  - completare **`Stanza`** ed il suo metodo **`removeAttrezzo()`**
  - utilizzare il codice di **`Borsa`** riportato di seguito completando il metodo **`removeAttrezzo()`**
  - modificare **`DiaDia`** implementando il codice per l'esecuzione dei comandi **`prendi`** e **`posa`**
    - gli attrezzi presi vengono rimossi dalla stanza e aggiunti alla borsa
    - gli attrezzi posati vengono rimossi dalla borsa e aggiunti alla stanza
  - la sintassi per inserire questi comandi è la seguente:
    - **`prendi <nomeAttrezzo>`**
    - **`posa <nomeAttrezzo>`**
  - modificare la logica del comando «**`aiuto`**» che deve tener conto del nuovo comando disponibile

Classe Borsa: Codice (1)

```
public class Borsa {
    public final static int DEFAULT_PESO_MAX_BORSA = 10;
    private Attrezzo[] attrezzi;
    private int numeroAttrezzi;
    private int pesoMax;

    public Borsa() {
          this(DEFAULT_PESO_MAX_BORSA);
    }

    public Borsa(int pesoMax) {
          this.pesoMax = pesoMax;
          this.attrezzi = new Attrezzo[10]; // speriamo bastino...
          this.numeroAttrezzi = 0;
    }

    public boolean addAttrezzo(Attrezzo attrezzo) {
          if (this.getPeso() + attrezzo.getPeso() > this.getPesoMax())
                return false;
          if (this.numeroAttrezzi==10)
                return false;
          this.attrezzi[this.numeroAttrezzi] = attrezzo;
          this.numeroAttrezzi++;
          return true;
    }

    public int getPesoMax() {
          return pesoMax;
    }

    public Attrezzo getAttrezzo(String nomeAttrezzo) {
          Attrezzo a = null;
          for (int i= 0; i<this.numeroAttrezzi; i++)
                if (this.attrezzi[i].getNome().equals(nomeAttrezzo))
                      a = attrezzi[i];
          return a;
    }
```

(continua)

Classe Borsa: Codice (2)

```
    public int getPeso() {
            int peso = 0;
            for (int i= 0; i<this.numeroAttrezzi; i++)
                        peso += this.attrezzi[i].getPeso();
            return peso;
    }

    public boolean isEmpty() {
            return this.numeroAttrezzi == 0;
    }

    public boolean hasAttrezzo(String nomeAttrezzo) {
            return this.getAttrezzo(nomeAttrezzo)!=null;
    }


    public Attrezzo removeAttrezzo(String nomeAttrezzo) {
            Attrezzo a = null;
            // --->  TODO (implementare questo metodo) <---
            return a;
    }

    public String toString() {
            StringBuilder s = new StringBuilder();
            if (!this.isEmpty()) {
                        s.append("Contenuto borsa ("+this.getPeso()+"kg/"+this.getPesoMax()+"kg): ");
                        for (int i= 0; i<this.numeroAttrezzi; i++)
                                    s.append(attrezzi[i].toString()+" ");
            }
            else
                        s.append("Borsa vuota");
            return s.toString();
    }

}
```

Esercizio 4 (Package)

- Il progetto sta crescendo: organizziamo meglio le classi in package
  - mettere **`Labirinto`** e **`Stanza`** nel package `it.uniroma3.diadia.ambienti`
  - mettere **`Attrezzo`** nel package `it.uniroma3.diadia.attrezzi`
  - mettere **`Giocatore`** e **`Borsa`** nel package `it.uniroma3.diadia.giocatore`
  - mettere **`Comando`**, **`DiaDia`** e **`Partita`** nel package `it.uniroma3.diadia`

Esercizio 5 (Disaccoppiamento I/O)

- La gestione dell'Input e dell'Output è attualmente "disseminata" in molti punti del codice
  - uso diretto di **`System.out`** (per la stampa su video)
  - uso diretto di **`System.in`** (per la lettura da tastiera)
- Questa situazione non è ottimale
  - difficile trovare e cambiare i messaggi stampati
  - dentro un test-case risulta poco agevole
    - sia controllare le stampe con delle asserzioni
    - sia iniettare dei comandi scelti nel test stesso
  - sarà difficile cambiare la modalità di interazione con l'utente (ad es. per passare ad una modalità alternativa basata sull'utilizzo di GUI)

Esercizio 5

- Riorganizziamo completamente la gestione dell'I/O, disaccoppiando il gioco dall'uso diretto e pervasivo di **`System.out/System.in`**, operando come segue:
  - Introduciamo la classe **`it.uniroma3.diadia.IOConsole`** (fornita nella prossima slide) che centralizza l'accesso a **`System.out/System.in`**
  - Creiamo <u>una sola istanza</u> di questa classe nell'unico metodo **`main()`** dell'intero gioco di ruolo **`DiaDia.main()`**
  - Rifattorizziamo tutto il codice per far arrivare tale unica istanza ove serve
    - Può essere necessario aggiungere costruttori e/o modificare quelli già esistenti in alcune classi
  - Eliminiamo tutte le stampe e le letture dirette dal resto del codice rimpiazzandole con un appropriato uso dei metodi della classe **`IOConsole`**

Esercizio 5: IOConsole - ATTENZIONE

- NON può essere modificato il codice di **IOConsole**
- NON può essere modificato il suo package
- Crearne UNA SOLA ISTANZA in tutto il codice nel metodo **DiaDia.main()**
- NON è più possibile usare **System.in** e **System.out** altrove

```
package it.uniroma3.diadia;
import java.util.Scanner;

public class IOConsole {

  public void mostraMessaggio(String msg) {
    System.out.println(msg);
  }

  public String leggiRiga() {
    Scanner scannerDiLinee = new Scanner(System.in);
    String riga = scannerDiLinee.nextLine();
    scannerDiLinee.close();
    return riga;
  }
}
```

Per il momento omettere questa riga, ci torneremo

TERMINI E MODALITA‘ DI CONSEGNA

- Svolgere in gruppi di <u>esattamente 2 persone</u>
- La soluzione deve essere consegnata al docente entro le 14:00 di mercoledì 8 aprile 2026 tramite GitHub
- Modalità di consegna
  - Creare un repository Github
  - Svolgere l’homework sul proprio calcolatore
  - Caricare il sorgente sviluppato
  - Creare una release di nome **`HWA`**
  - Compilare la form a questo indirizzo:

    https://forms.gle/AHp3RFXD1tJBnm3L7

    - Iscriversi a PI1 compilando una form con i propri dati
    - Includere un link alla release del sorgente del codice

MODALITA‘ DI CONSEGNA

- Non inviare mail di richiesta di conferma di avvenuta ricezione, non saranno gestite
  - Eventuali problematiche saranno gestite successivamente, quando e se *veramente* serve
- ✔ Consegne multiple da evitare ed inutili
  - ✔ Inutile: fa fede la data della release non della compilazione della form online
  - ✔ Ma solo i dati raccolti con la form online verranno gestite automaticamente per raccogliere le matricole di chi ha consegnato

TERMINI E MODALITA‘ DI CONSEGNA

- ATTENZIONE:
  - Senza l'invio di questo homework, *non* sarà possibile continuare il percorso HQ di questo anno accademico

Consegna con GitHub (1)

- GitHub è un servizio di hosting di progetti versionati con *git*
  - Darà ospitalità ai sorgenti dei nostri progetti
- Principali obiettivi del servizio
  - permettere la cooperazione di più sviluppatori condividendo i sorgenti dello stesso progetto
  - conservare la storia del progetto e di tutte le versioni dei sorgenti mai prodotte

  Qualche riferimento (fate anche qualche ricerca…):
  - *https://github.com/git-guides*

    (ma si trovano anche interi libri…)
- L'obiettivo è <u>*cominciare*</u> ad usare *git*
  - per ora solo uno strumento che faciliti le consegne
  - nel futuro, strumento *abilitante* il lavoro cooperativo, in team
- L'utilizzo efficace di git *NON* è tra gli obiettivi formativi del corso

  ✔ Il suo utilizzo è fortemente *incentivato…*

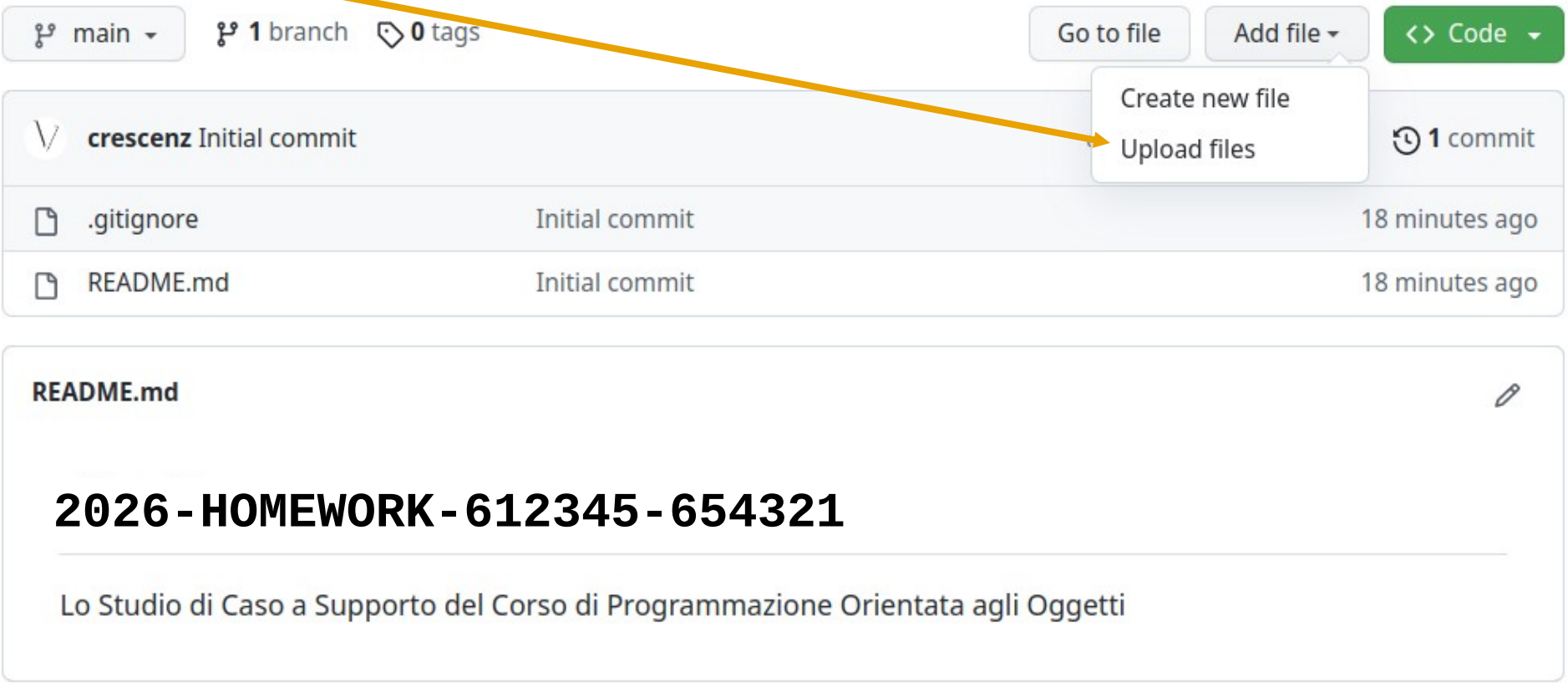
  …ma *richiesto* solo come strumento per la consegna

Consegna con GitHub (2)

- Caricare il progetto su GitHub. Faremo una prima *release* (>>)
  - Registrarsi su *github.com*
  - Creare un nuovo repository (pubblico) per ospitare il codice del vostro primo progetto
    - cliccando su «create repository»
  - Il nome della repository dovrà essere
    - `2026-HOMEWORK-«MATRICOLA1»-«MATRICOLA2»`
    - ad es.: `2026-HOMEWORK-612345-654321`
  - *Git Repository*
    - il luogo dove i file del sorgente del progetto sono “versionati”
      - ovvero viene conservata traccia di tutta la loro «storia» dal momento in cui sono caricati all’interno del repository ad oggi
    - accessibile da altri utenti e modificabile dagli altri *collaboratori*

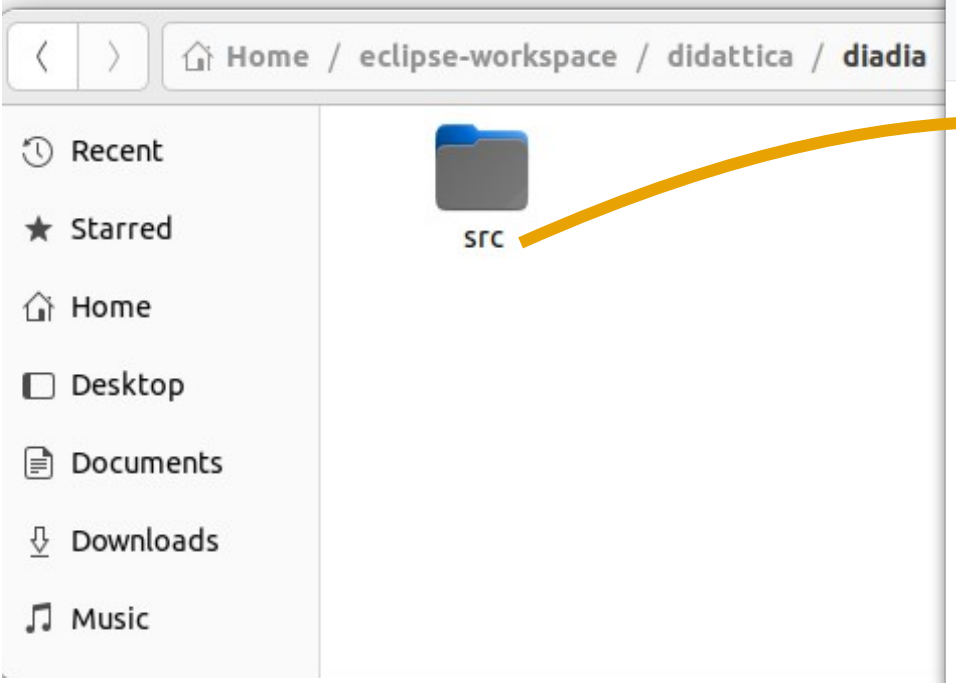
Consegna con GitHub (3)

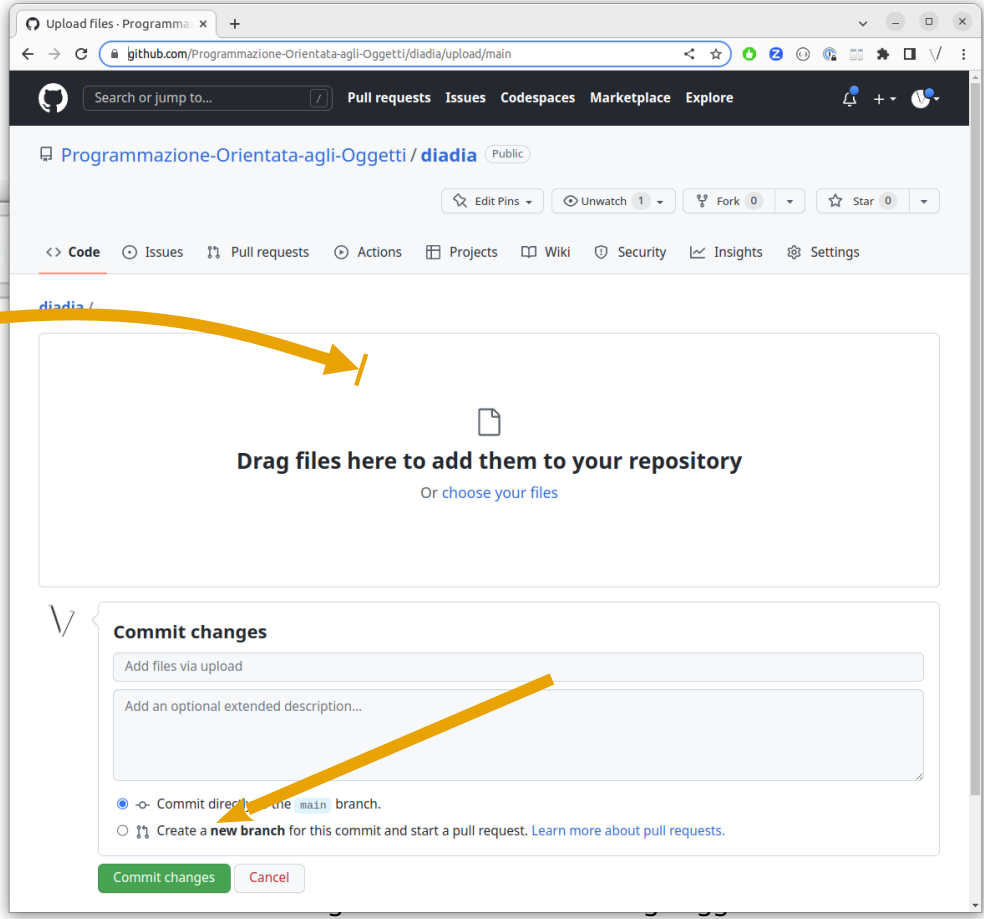
- Una volta creato il repository, caricare il codice sorgente del progetto seguendo le istruzioni che leggete su schermo, oppure, ancora più semplicemente:
  - Nella pagina della repository appena creata, utilizzare: *upload files*



Consegna con GitHub (4)

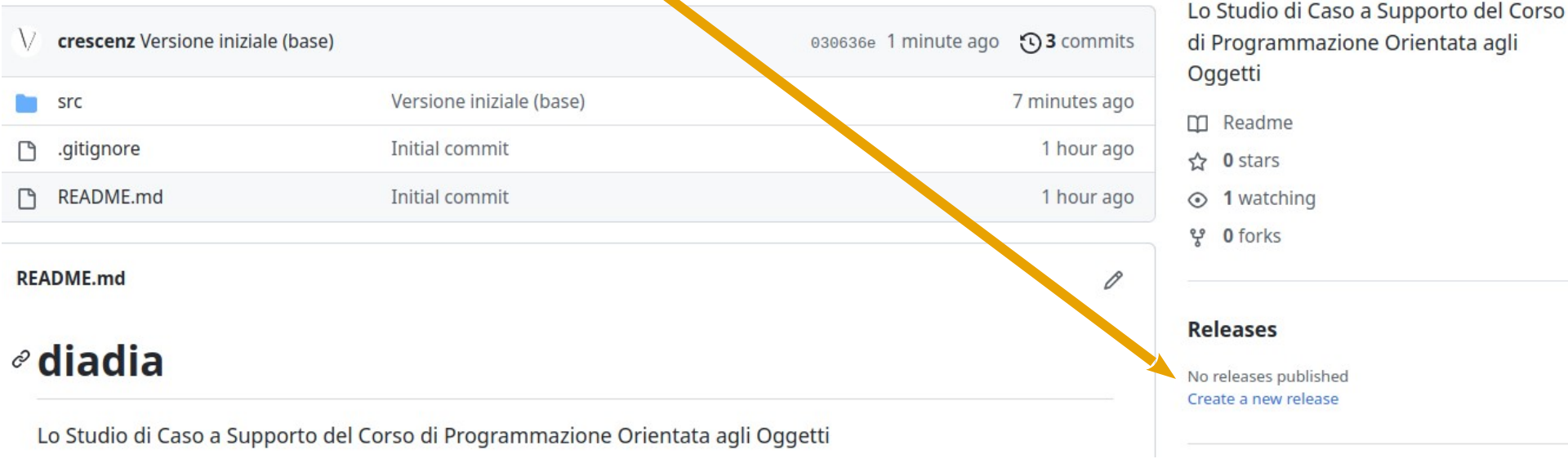
- Navigare fino alla directory del vostro progetto
  - ad es. `/eclipse-workspace/didattica/diadia/`
  - Trascinare *tutta* la cartella `src` dentro il git repository appena creato
    - Successivamente ripetere l'operazione con la cartella `test`
- Quindi salviamo le modifiche
  - tasto verde *Commit changes*





Consegna con GitHub (5)

- Per finire, creare una *release*: “impacchettamento” del progetto per renderlo disponibile ad altri
  - Selezionare <<create new release>> a dx nella pagina



  - Scegliere il cosidetto *tag* della release
    - Usare come tag: HWA
    - Come nome della release inserire
    - `2026-HOMEWORKA <<matricola1>> <<matricola2>>`
      - Ad es. `2026-HOMEWORKA 612345 654321`
  - Nella descrizione scrivere eventuali malfunzionamenti noti ma non risolti

TERMINI E MODALITA' DI CONSEGNA

- Esempio di come appare la release della versione base pubblicata
- ✔ Copiare l'URL dell'archivio (NON l'archivio) ed incollarlo nella form come di seguito spiegato

CONSEGNA

https://forms.gle/AHp3RFXD1tJBnm3L7

Percorso HQ 2025-2026

Iscrizione PI1 e Consegna HWA
Termine ultimo per l'iscrizione alla prova: MERCOLEDI 8 aprile ore 14.00.
La prova è organizzata in turni che si terranno venerdì 10 aprile a partire dalle ore 8.45 circa in laboratorio Campus One. I turni saranno comunicati nel pomeriggio di mercoledì sul gruppo del corso.

Next Page 1 of 2 Clear form

Versione base

Pre-release

crescenz released this 2 minutes ago v0.0.0 030636e

La versione iniziale (base) del codice

▾ Assets 2

Source code (zip)
Source code

Open link in new tab
Open link in new window
Open link in incognito window
Save link as...
Copy link address
1Password – Password Manager
AdBlock — best ad blocker
Inspect

© 2023 GitHub, Inc. Status

Percorso HQ 2025-2026

* Indicates required question

Per iscriversi, fornire le seguenti informazioni

Le informazioni raccolto saranno utilizzate sole ed esclusivamente a supporto dell'organizzazione del percorso HQ di quest'anno accademico, ed a supporto di attività di miglioramento della didattica del corso
(I campi marcati con * sono obbligatori. Se ancora non si possiede una matricola, cortesemente ripetere il cognome.)

Nome *
Your answer

Cognome *
Your answer

Matricola *
Your answer

Indirizzo url del sorgente con tag HWA *
Your answer

Eventuali malfunzionamenti noti ma non risolti
Your answer